



FeedbackPro

Retours anonymes pour l'amélioration continue
des services publics et privés

Documentation technique et conceptuelle
Application mobile Flutter - Plateforme web Next.js - Supabase

Espace admin identique sur mobile et web - Offline-first - Multilingue mg / fr / en

Contexte : Madagascar et Afrique
Donner de la voix, améliorer les services



Table

	5 1. Introduction
1.1 Objectifs du document	5
1.2 Public vise	5
1.3 Identite visuelle	5
	6 2. Analyse des besoins
2.1 Besoins des utilisateurs (citoyens)	6
2.2 Besoins des responsables (admins)	6
2.3 Besoins fonctionnels transverses	6
	7 3. Architecture
3.1 Anonymat maximal	7
3.2 Materiel modeste et faible bande passante	7
3.3 Fiabilite des donnees	7
3.4 Securite et roles	7
	8 4. Analyse de faisabilite
	9 5. Plan de developpement
5.1 Une seule application pour le citoyen	9
5.2 Effet de reseau et masse critique	9
5.3 Mutualisation technique et reduction des couts	9
5.4 Comparaison inter-secteurs et pilotage public	9
5.5 Adaptabilite par templates	9
	10 6. Architecture technique
6.1 Couches de l'application mobile	10
6.2 Espace admin identique mobile et web	10
	11 7. Conception
7.1 Diagramme de cas d'utilisation	11
7.2 Diagramme de classes (modele de donnees)	12
7.3 Diagramme de sequence (soumission d'un feedback)	12
	13 8. Synthese
8.1 Flux de synchronisation	13



8.2 Idempotence et absence de doublon	13
8.3 Gestion des conflits	13
8.4 Reprise et robustesse	13
9. Se	14
9.1 Anonymat par conception	14
9.2 Regles de securite au niveau des lignes (RLS)	14
9.3 Roles d'administration	14
9.4 Compte administrateur	14
10. C	15
10.1 Fonctionnalites livrees	15
10.2 Fonctionnalites avancees et extensions	16
11. D	17
11.1 Vue de deploiement	17
11.2 Mise en route de l'admin web	17
11.3 Base de donnees	17
12. F	18
13. C	19





1. Introduction

FeedbackPro est une solution complete de collecte et de gestion de retours (feedbacks) totalement anonymes. Elle permet a tout citoyen de donner son avis sur n'importe quel service - ecole, hopital, clinique, boutique, restaurant, hotel, administration publique, transport - et aux responsables de suivre, analyser et agir rapidement grace a une plateforme web professionnelle et a un espace d'administration identique dans l'application mobile.

Ce palier apporte une evolution majeure : un formulaire intelligent avec validation en temps reel et logique conditionnelle forte selon le secteur et la note, une synchronisation locale-serveur fiable avec gestion des conflits, une recherche intelligente dont les resultats s'affichent directement dans l'accueil, un indicateur clair de la page active, un espace admin ou le responsable peut tout faire (moderation, publication, export, gestion des utilisateurs), et un logo de marque decline partout (splash, icone, en-tete, admin web).

1.1 Objectifs du document

Ce document presente l'analyse des besoins et des contraintes, justifie les choix technologiques et conceptuels, decrit la conception UML du systeme, detaille l'architecture et la synchronisation, catalogue l'ensemble des fonctionnalites, et conclut sur la valeur apportee et la feuille de route.

1.2 Public vise

Equipe de developpement, responsables d'etablissements, decideurs publics, ainsi que toute partie prenante interessee par l'amelioration continue des services par la voix citoyenne.

1.3 Identite visuelle

Le logo FeedbackPro combine une bulle de dialogue (le retour citoyen) et des barres ascendantes (la progression de la satisfaction), rehaussees d'une pastille doree d'evaluation. La couleur de marque est le vert (#1B7A43), symbole de confiance et de progres. Le logo est decline sur l'ecran de demarrage, l'icone d'application, les en-tetes et l'admin web.



2. Analyse des besoins

2.1 Besoins des utilisateurs (citoyens)

- Soumettre un feedback totalement anonyme, sans créer de compte.
- Identifier l'établissement concerné grâce à une recherche intelligente.
- Noter un service (étoiles, échelle, smileys).
- Choisir un type de problème adapté au secteur.
- Joindre des photos ou une courte vidéo, et la localisation GPS si utile.
- Accéder rapidement à un établissement via un QR code.
- Suivre l'avancement de son feedback et dialoguer de manière anonyme.
- Utiliser l'application même sans connexion (offline-first).
- Choisir sa langue : malgache, français ou anglais.

2.2 Besoins des responsables (admins)

- Visualiser et filtrer tous les feedbacks par secteur, note, statut, date.
- Être alerte immédiatement en cas de feedback critique.
- Moderer, prioriser, changer le statut et publier des actions correctives.
- Gérer les établissements et les améliorations (créer, modifier, supprimer).
- Analyser les tendances : positifs / négatifs, évolution, comparaisons.
- Exporter les données (CSV, Excel, PDF) et garder un historique d'audit.
- Gérer plusieurs administrateurs avec des rôles distincts.
- Disposer du même espace admin sur mobile et sur le web.

2.3 Besoins fonctionnels transverses

Validation et contraintes robustes en temps réel (mobile et web), gestion intelligente de la logique des formulaires, transparence via des statistiques publiques anonymisées, recherche instantanée dans l'accueil, interface totalement responsive et notifications push pertinentes.



3. Analyse des contraintes

3.1 Anonymat maximal

Aucune donnée permettant d'identifier l'utilisateur ne doit être stockée. La table des feedbacks ne contient aucun identifiant utilisateur : le suivi se fait via un code anonyme aléatoire connu du seul auteur. Les règles de sécurité au niveau des lignes (RLS) garantissent qu'un utilisateur ne peut jamais lire les feedbacks d'autrui.

3.2 Matériel modeste et faible bande passante

La cible inclut des téléphones Android d'entrée de gamme et des connexions instables. D'où : compression agressive des images, cartes OpenStreetMap légères, interface sobre, et surtout une architecture offline-first ou tout fonctionne hors ligne puis se synchronise automatiquement.

3.3 Fiabilité des données

Les règles métier sont appliquées à deux niveaux : côté client (validation temps réel dans le formulaire, avec logique conditionnelle selon secteur et note) et côté serveur (triggers PostgreSQL). La synchronisation est idempotente (identifiant client unique) et gère les conflits, garantissant la cohérence de bout en bout.

3.4 Sécurité et rôles

Authentification anonyme sur mobile, mais email / mot de passe pour les admins, avec gestion de rôles (Super Admin, Admin de domaine, Modérateur) et journal d'audit de toutes les actions sensibles. Le compte administrateur de référence est `luciarasoanirina8@gmail.com`.



4. Arguments du choix des technologies

Flutter (mobile)	Un seul code pour Android / iOS, performances natives, riche ecosysteme, ideal pour une UI moderne et fluide sur appareils modestes.
Supabase (backend)	PostgreSQL manage + Auth + Storage + Realtime + Edge Functions. La securite par RLS est parfaite pour garantir l'anonymat. Open-source, evite le verrouillage propriétaire.
Isar (local)	Base locale rapide et typee pour le mode offline-first ; lecture / ecriture instantanees sans connexion, synchronisation differee.
Riverpod	Gestion d'etat moderne, testable, en couches ; rend le changement de langue et de theme instantane.
Next.js 15 (web)	App Router + rendu serveur pour un tableau de bord rapide et securise ; TypeScript pour la fiabilite, Tailwind pour une UI soignee et responsive, recharts pour des graphiques interactifs.
flutter_map + OSM	Cartographie libre, sans cle API ni cout, et legere en bande passante - adaptee au contexte local.

Ces choix forment une pile coherente, moderne, economique et adaptee aux contraintes de terrain (cout, connectivite, materiel).



5. Pourquoi couvrir tous les domaines plutôt qu'un seul

Un choix structurant est de couvrir l'ensemble des secteurs (santé, éducation, commerce, administration, restauration / hôtellerie, transport) plutôt que de se limiter à un seul domaine. Les raisons :

5.1 Une seule application pour le citoyen

Le citoyen utilise une multitude de services au quotidien. Une application unique, multi-domaines, évite d'installer dix applications différentes et augmente fortement l'adoption et la fréquence d'usage.

5.2 Effet de réseau et masse critique

Regrouper tous les secteurs crée une masse critique d'utilisateurs et de données. Plus il y a de retours, plus les statistiques sont fiables et plus la plateforme devient utile pour tous - un cercle vertueux qu'un produit mono-domaine atteint difficilement.

5.3 Mutualisation technique et réduction des coûts

L'infrastructure (anonymat, offline, synchronisation, modération, cartes, exports) est commune à tous les secteurs. La mutualiser réduit fortement le coût de développement et de maintenance, tout en assurant une expérience homogène.

5.4 Comparaison inter-secteurs et pilotage public

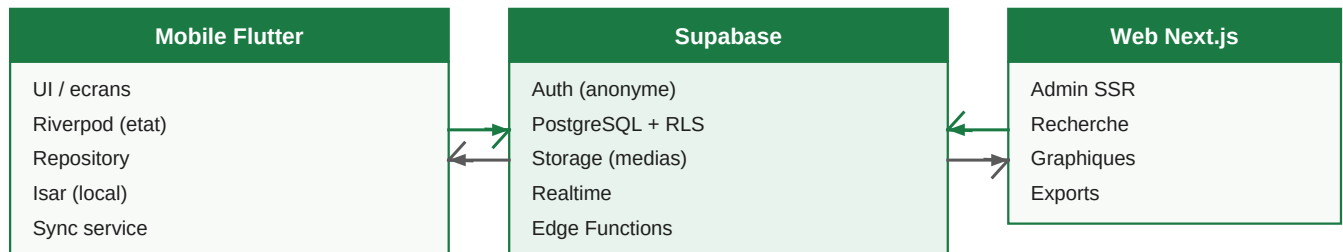
Couvrir tous les domaines permet des analyses transversales (comparer par exemple la satisfaction entre hôpitaux et administrations d'une même région) - une valeur stratégique pour les décideurs publics, impossible avec une approche cloisonnée.

5.5 Adaptabilité par templates

La généralité ne nuit pas à la pertinence : des templates de questionnaires spécifiques par secteur, et un formulaire à logique conditionnelle, adaptent l'expérience à chaque domaine tout en gardant un socle commun.

6. Architecture generale

FeedbackPro s'organise en trois grands blocs : le client mobile Flutter (offline-first), le backend Supabase (source de verite), et la plateforme web Next.js. L'espace admin est disponible a la fois dans le mobile et sur le web, avec le meme design et les memes fonctionnalites.



Le mobile écrit d'abord en local (Isar) puis synchronise vers Supabase ; l'admin mobile et l'admin web lisent la même source de vérité en quasi temps réel (Realtime).

6.1 Couches de l'application mobile

Présentation (écrans + widgets), état (Riverpod), domaine (entités et interfaces), données (repositories offline-first), sources (Isar local et Supabase distant). Chaque couche ne dépend que de la couche inférieure, ce qui rend le code testable et maintenable.

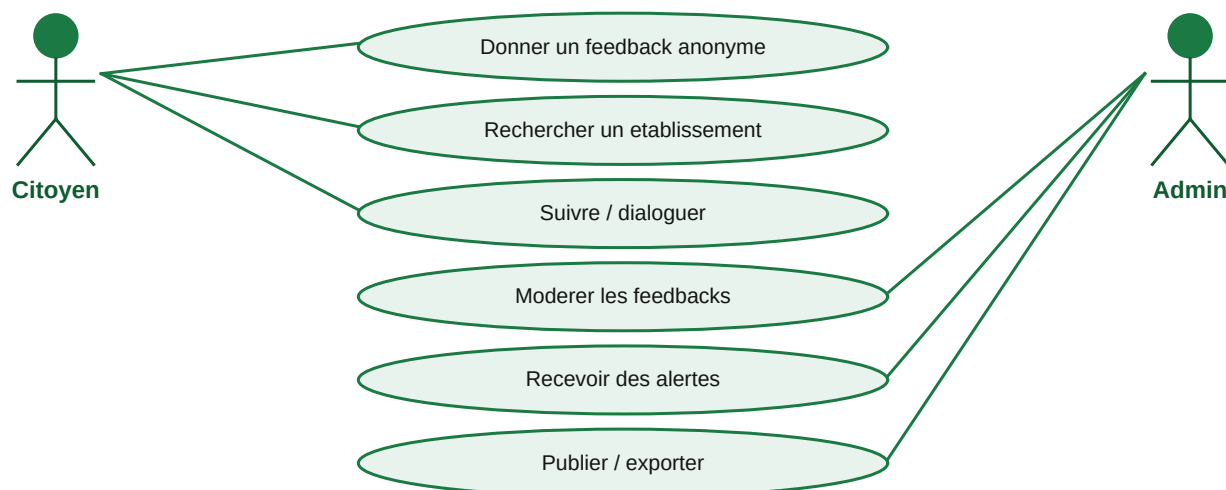
6.2 Espace admin identique mobile et web

L'espace admin partage la même charte (logo, vert de marque, indicateur de page active) et les mêmes capacités : modération complète, publication d'améliorations, gestion des établissements et des alertes, export et gestion des utilisateurs. Sur mobile il est intégré à l'application ; sur le web il s'appuie sur Next.js et reste totalement responsive.

7. Conception UML

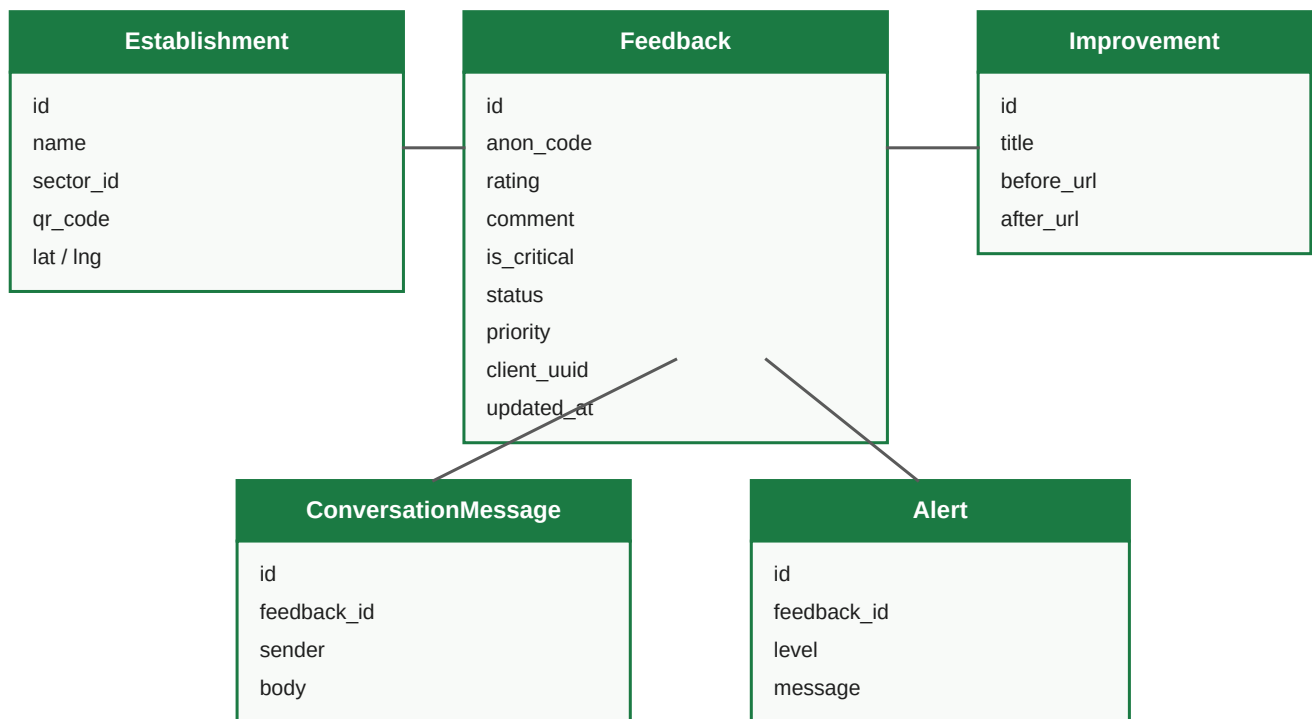
7.1 Diagramme de cas d'utilisation

Acteurs principaux et leurs interactions avec le systeme :



Le citoyen donne des feedbacks, recherche des etablissements et suit / dialogue de facon anonyme. L'admin modere, recoit les alertes, publie les actions correctives et exporte les donnees.

7.2 Diagramme de classes (modele de donnees)



Le feedback est l'entite centrale (sans aucun lien d'identite). Il se rattache a un etablissement, peut generer des messages de conversation, des alertes, et donner lieu a des ameliorations publiees. Les champs client_uuid et updated_at soutiennent une synchronisation idempotente et la resolution des conflits.

7.3 Diagramme de sequence (soumission d'un feedback)

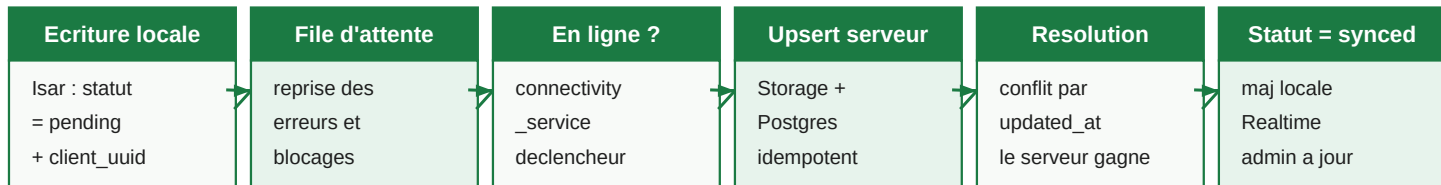
- 1. L'utilisateur recherche l'etablissement et remplit le formulaire (validation temps reel, logique selon secteur et note).
- 2. A l'envoi, le feedback est ecrit immediatement dans Isar (local) avec un client_uuid unique et le statut en attente de synchronisation.
- 3. Si en ligne : upload des medias compressees vers Supabase Storage.
- 4. Upsert de la ligne dans PostgreSQL (RLS : insertion anonyme, idempotence par client_uuid).
- 5. Les triggers valident, creent une alerte si critique, indexent (FTS).
- 6. L'admin (mobile et web) voit le feedback et l'alerte en quasi temps reel (Realtime).



8. Synchronisation Isar <-> Supabase

La synchronisation est le coeur de la fiabilité offline-first. Elle est conçue pour ne jamais perdre de donnée, ne jamais créer de doublon, et résoudre proprement les conflits entre l'état local et l'état serveur.

8.1 Flux de synchronisation



8.2 Idempotence et absence de doublon

Chaque feedback porte un identifiant client (client_uuid) généré sur l'appareil. L'écriture serveur est un upsert sur cette clé : rejouer une synchronisation (après coupure réseau, crash ou reprise au démarrage) n'insère jamais deux fois la même donnée.

8.3 Gestion des conflits

Lorsqu'une ligne existe déjà des deux côtés, la résolution s'appuie sur l'horodatage updated_at. Règle par défaut : le serveur fait autorité pour les champs de modération (statut, priorité), tandis que le contenu d'origine du citoyen reste immuable. Les champs modifiés localement mais non encore poussés sont rejoués ; en cas d'écart, la valeur la plus récente l'emporte, et l'événement est tracé dans le journal d'audit.

8.4 Reprise et robustesse

Une synchronisation est déclenchée au démarrage, au retour de la connectivité et après chaque envoi. Les feedbacks en erreur ou bloqués sont systématiquement repris. Aucune action utilisateur n'est requise : le système converge vers un état cohérent dès que le réseau le permet.

Correctif appliqué : les feedbacks en erreur ou bloqués sont re-synchronisés automatiquement, la synchro s'exécute au démarrage, et l'idempotence par client_uuid supprime tout risque de doublon.



9. Securite, anonymat et roles

9.1 Anonymat par conception

La table `feedbacks` ne stocke aucun identifiant utilisateur. Le suivi et le dialogue passent par un code anonyme aleatoire (`anon_code`) connu du seul auteur. La conversation 2-way s'obtient via la fonction serveur `get_conversation(anon_code)`.

9.2 Regles de securite au niveau des lignes (RLS)

- Insertion ouverte aux sessions anonymes (le citoyen peut déposer un retour).
- Lecture et moderation reservees aux comptes admins declares.
- Un utilisateur ne peut jamais lire les feedbacks d'autrui.
- Les buckets de medias appliquent les memes principes d'accès.

9.3 Roles d'administration

Trois roles distincts : Super Admin (pilotage global, gestion des administrateurs), Admin de domaine (perimetre sectoriel) et Modérateur (traitement des feedbacks). Chaque action sensible est enregistree dans un journal d'audit horodate.

9.4 Compte administrateur

Le compte de reference est `luciarasoanirina8@gmail.com`. Il se cree dans Supabase Auth (email + mot de passe, auto-confirme), puis on execute `supabase/setup_admin.sql` pour l'habiliter. Le mobile et le web pointent sur le meme projet Supabase afin de partager les memes donnees.



10. Catalogue des fonctionnalités

10.1 Fonctionnalités livrées

Logo de marque partout splash, icône, en-têtes, admin web et mobile.

Espace admin identique même design et mêmes fonctions sur mobile et web, responsive.

Synchronisation fiable Isar <-> Supabase, idempotente, avec gestion des conflits.

Formulaire intelligent validation temps réel et logique conditionnelle selon secteur et note.

Recherche intelligente résultats affichés directement dans l'accueil (établissements + secteurs).

Indicateur de page active repérage visuel clair de la page en cours, mobile et web.

Admin peut tout faire modération complète, publication, export, gestion des utilisateurs.

Interface responsive adaptée à tous les appareils, du téléphone au grand écran.

Offline-first usage complet hors ligne, synchronisation automatique différée.

Multilingue instantané malgache, français, anglais - changement immédiat.

Statistiques et tendances graphiques interactifs, comparaisons, évolution.

QR code établissement accès direct au bon établissement par scan.



10.2 Fonctionnalités avancées et extensions

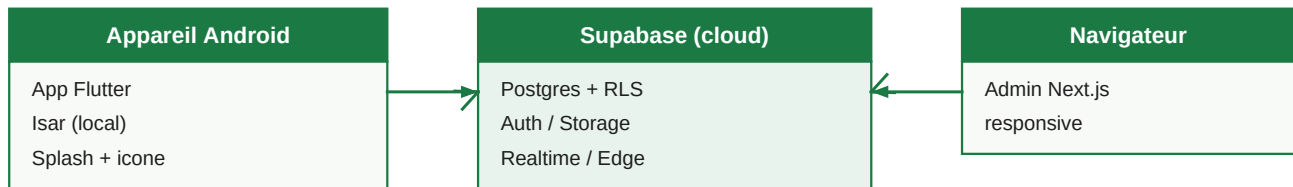
Ces fonctionnalités complètent le socle et enrichissent progressivement la plateforme, en s'appuyant sur les Edge Functions Supabase et les services mobiles.

Analyse IA	Edge Function d'analyse de sentiment et de thèmes ; remplit les champs sentiment et thèmes des feedbacks.
Notifications push	FCM + Edge Function pour notifier réponses et améliorations.
Conversations 2-way	échange anonyme citoyen <-> admin via get_conversation.
Export complet	CSV, Excel et PDF détaillé de plus de dix pages.
Support vocal	saisie et restitution vocales pour l'accessibilité.
Traduction automatique	des retours entre malgache, français et anglais.
Alertes intelligentes	déclenchement automatique sur feedback critique.
Suivi du statut	reçu, en cours, résolu - visible par le citoyen.
Historique d'audit	journal horodaté de toutes les actions sensibles.
Heatmap géolocalisée	carte de chaleur des problèmes (flutter_map + clustering).

Toutes ces fonctionnalités sont pensées pour le contexte local : faible bande passante, matériel modeste, pluralité linguistique et exigence forte d'anonymat.

11. Deploiement et exploitation

11.1 Vue de deploiement



L'application mobile se distribue en APK (arm32 / arm64) ou via un store. Le backend Supabase est hébergé dans le cloud (projet unique partagé). L'admin web se build avec Next.js puis se sert en production (Node) ou se déploie sur une plateforme statique / edge.

11.2 Mise en route de l'admin web

- Renseigner `web_admin/.env.local` (URL et cle anon du projet Supabase).
- Installer les dependances : `npm install`.
- Construire : `npm run build` (le logo est integre a l'interface).
- Servir : `npm start`, puis se connecter avec le compte administrateur.

11.3 Base de donnees

Executer les scripts SQL dans l'ordre (init du schema et RLS, buckets de stockage, evolutions), puis creer le compte admin et lancer `setup_admin.sql`. Des enregistrements de demonstration realistes et coherents peuvent etre charges pour illustrer l'usage.



12. Feuille de route

La solution est concue pour evoluer par paliers successifs, sans rupture d'usage :

- Generalisation de l'analyse IA (priorisation automatique, resume des themes).
- Notifications push contextualisees et silencieuses hors heures utiles.
- Heatmap temps reel et tableaux de bord territoriaux pour les decideurs.
- Recompenses optionnelles via Mobile Money (MVola / Orange / Airtel).
- Ouverture d'API publiques anonymisees pour la transparence citoyenne.
- Accessibilite renforcee : support vocal complet et contrastes eleves.

Chaque palier reste fidele aux principes fondateurs : anonymat par conception, offline-first, fiabilite des donnees et sobriete technique.



13. Conclusion

FeedbackPro répond à un besoin concret : donner une voix anonyme et sûre aux citoyens, et des outils d'action efficaces aux responsables, dans un contexte de connectivité et de matériel contraints.

Les choix d'architecture - anonymat par conception, offline-first, synchronisation fiable avec gestion des conflits, règles métier appliquées à la fois côté client et serveur, pile technologique moderne et économique - garantissent fiabilité, sécurité et adoption. L'approche multi-domaines maximise l'impact social et la valeur des données.

Avec un espace admin identique sur mobile et web, une recherche intelligente intégrée à l'accueil, un formulaire à logique conditionnelle forte et un logo de marque décliné partout, FeedbackPro s'affirme comme un outil de référence pour l'amélioration continue des services.



FeedbackPro - Donner de la voix, améliorer les services.